

**UNIVERSIDAD DEL VALLE**

Facultad de Ingeniería

Escuela de Ingeniería de Sistemas y Computación

# **RouteFlow SaaS**

Arquitectura de Software

**Entregable de Arquitectura de Software**

**Proyecto:** Sistema de gestión y optimización de rutas logísticas

## **Integrantes del equipo:**

Juan David Cuellar Lopez

Juan David Perez Valencia

Joseph David Herrera Libreros

Kevin Alexis Lorza Ramirez

Samuel Escobar Rivera

29 de septiembre de 2026

# Índice

|                                                                                         |          |
|-----------------------------------------------------------------------------------------|----------|
| <b>1. Contexto</b>                                                                      | <b>2</b> |
| <b>2. Contenedores Principales</b>                                                      | <b>2</b> |
| 2.1. Frontend Responsive . . . . .                                                      | 2        |
| 2.1.1. Vista de Administración y Despacho . . . . .                                     | 3        |
| 2.1.2. Vista de Conductor . . . . .                                                     | 3        |
| 2.2. Backend API – Servidor de Aplicaciones . . . . .                                   | 3        |
| 2.3. Motor Geográfico y de Optimización . . . . .                                       | 4        |
| 2.4. Base de Datos Principal . . . . .                                                  | 4        |
| <b>3. Módulos del Sistema</b>                                                           | <b>4</b> |
| 3.1. Módulo de Autenticación y Multi-tenancy . . . . .                                  | 4        |
| 3.2. Módulo de Gestión de Flota y Bodegas . . . . .                                     | 4        |
| 3.3. Módulo de Carga de Datos y Geocodificación . . . . .                               | 4        |
| 3.4. Módulo de Optimización de Rutas . . . . .                                          | 4        |
| 3.5. Módulo de Monitoreo y Ejecución . . . . .                                          | 5        |
| <b>4. Responsabilidad de Cada Módulo</b>                                                | <b>5</b> |
| <b>5. Dependencias Permitidas</b>                                                       | <b>5</b> |
| 5.1. Frontend Responsive y Backend . . . . .                                            | 5        |
| 5.2. Backend y Base de Datos . . . . .                                                  | 6        |
| 5.3. Backend y Motor de Ruteo . . . . .                                                 | 6        |
| <b>6. Contrato de una Operación</b>                                                     | <b>6</b> |
| 6.1. Operación . . . . .                                                                | 6        |
| 6.2. Cuerpo de Entrada . . . . .                                                        | 6        |
| 6.3. Cuerpo de Salida . . . . .                                                         | 6        |
| <b>7. ADR-001 – Selección de PostgreSQL con PostGIS</b>                                 | <b>7</b> |
| <b>8. Walking Skeleton</b>                                                              | <b>7</b> |
| 8.1. Paso 1 – Autenticación y configuración inicial . . . . .                           | 7        |
| 8.2. Paso 2 – Carga de entregas . . . . .                                               | 7        |
| 8.3. Paso 3 – Geocodificación y ejecución del contrato de optimización . . . . .        | 8        |
| 8.4. Paso 4 – Visualización y retroalimentación en tiempo real por WebSockets . . . . . | 8        |
| 8.5. Flujo completo del Walking Skeleton . . . . .                                      | 8        |
| <b>9. Resumen Arquitectónico</b>                                                        | <b>8</b> |

# 1. Contexto

**RouteFlow** es un sistema SaaS de gestión y optimización de rutas orientado principalmente a pequeñas y medianas empresas de distribución, comercios electrónicos locales y establecimientos que realizan entregas a domicilio.

El objetivo principal del sistema es eliminar la necesidad de realizar manualmente el cálculo y la organización de rutas mediante herramientas de mapas convencionales. Para ello, RouteFlow permite gestionar múltiples entregas y vehículos desde un panel de control centralizado y generar rutas optimizadas en cuestión de minutos.

El sistema busca facilitar la operación logística mediante la integración de información relacionada con:

- Empresas y usuarios.
- Vehículos y capacidades de carga.
- Bodegas o depósitos.
- Entregas y paquetes.
- Direcciones y coordenadas geográficas.
- Ventanas horarias.
- Rutas optimizadas.
- Seguimiento del estado de las entregas.

La solución contempla una arquitectura orientada a servicios, en la cual una interfaz frontend responsive se comunica con un backend centralizado que administra la lógica de negocio, la persistencia de datos y la comunicación con servicios externos de geolocalización y ruteo.

## 2. Contenedores Principales

### 2.1. Frontend Responsive

Aplicación web responsive desarrollada utilizando tecnologías como React o Next.js. El frontend está diseñado para adaptarse a diferentes tamaños de pantalla, permitiendo que tanto administradores, despachadores como conductores puedan acceder al sistema desde computadores, tabletas o dispositivos móviles.

Dentro del mismo frontend se definen dos grupos principales de vistas:

### 2.1.1. Vista de Administración y Despacho

Está orientada principalmente a los usuarios con roles de Administrador y Despachador. Entre sus principales funcionalidades se encuentran:

- Administrar empresas y usuarios.
- Gestionar vehículos.
- Registrar y administrar bodegas.
- Cargar archivos CSV.
- Visualizar las entregas.
- Ejecutar la optimización de rutas.
- Consultar el estado de las operaciones.
- Visualizar información mediante mapas.

### 2.1.2. Vista de Conductor

Está orientada principalmente a los usuarios con rol de Conductor. Permite:

- Consultar la ruta asignada.
- Visualizar el orden de las paradas.
- Consultar información de las entregas.
- Actualizar el estado de las entregas.
- Consultar el progreso de la operación.
- Acceder a aplicaciones de navegación.

## 2.2. Backend API – Servidor de Aplicaciones

Servidor de aplicaciones desarrollado utilizando tecnologías como NestJS y Node.js. Constituye el núcleo de la solución y centraliza la lógica de negocio. Sus responsabilidades principales son:

- Autenticación y autorización.
- Gestión multi-tenant.
- Administración de usuarios y roles.
- Gestión de vehículos y bodegas.
- Gestión de entregas.

- Procesamiento de archivos CSV.
- Coordinación del proceso de optimización.
- Comunicación con la base de datos.
- Comunicación con servicios externos.
- Emisión de actualizaciones mediante WebSockets.

## 2.3. Motor Geográfico y de Optimización

Componente encargado de procesar la información geográfica y calcular las rutas óptimas. Este componente se comunica con **OSRM (Open Source Routing Machine)** para obtener matrices de distancias y tiempos basadas en la red vial. Posteriormente, utiliza algoritmos de optimización de problemas de ruteo de vehículos (*Vehicle Routing Problem*, VRP), mediante herramientas como **Google OR-Tools**.

## 2.4. Base de Datos Principal

La solución utiliza **PostgreSQL** como sistema gestor de bases de datos, complementado con la extensión **PostGIS** para el almacenamiento y procesamiento espacial de información geográfica, asegurando un estricto aislamiento multi-tenant por empresa.

# 3. Módulos del Sistema

## 3.1. Módulo de Autenticación y Multi-tenancy

Controla el acceso seguro al sistema mediante tokens JWT y garantiza la separación lógica de información entre las diferentes empresas.

## 3.2. Módulo de Gestión de Flota y Bodegas

Permite administrar los vehículos y las bodegas de origen de las rutas.

## 3.3. Módulo de Carga de Datos y Geocodificación

Permite importar masivamente entregas mediante archivos CSV y convertir direcciones en coordenadas geográficas.

## 3.4. Módulo de Optimización de Rutas

Procesa la información geográfica, genera la matriz de tiempos con OSRM y ejecuta los algoritmos VRP.

### 3.5. Módulo de Monitoreo y Ejecución

Permite realizar seguimiento en tiempo real de las operaciones y actualizar el estado de las entregas mediante WebSockets.

## 4. Responsabilidad de Cada Módulo

| Módulo           | Responsabilidad                                                                                                                                                |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Autenticación    | Gestionar el registro e inicio de sesión de usuarios mediante tokens JWT y controlar los permisos asociados a roles de Administrador, Despachador y Conductor. |
| Gestión de Flota | Crear, editar, consultar y administrar vehículos, incluyendo información como placas, estado y capacidad máxima expresada en kilogramos.                       |
| Carga de Datos   | Validar la estructura de archivos CSV que contienen información sobre direcciones, pesos y ventanas horarias de los paquetes.                                  |
| Geocodificación  | Convertir las direcciones de las entregas en coordenadas geográficas para permitir su posterior procesamiento espacial.                                        |
| Optimización     | Consumir los servicios de OSRM para obtener la matriz de tiempos y distancias y ejecutar algoritmos VRP determinando rutas óptimas.                            |
| Monitoreo        | Proporcionar actualizaciones en vivo mediante WebSockets sobre el progreso de la operación y registrar marcas de tiempo ( <i>timestamps</i> ).                 |

## 5. Dependencias Permitidas

### 5.1. Frontend Responsive y Backend

El Frontend Responsive se comunica exclusivamente con el Backend API mediante HTTP/REST para consultas y modificaciones, y WebSockets para actualizaciones en tiempo real. Las vistas no acceden directamente a la base de datos.

## 5.2. Backend y Base de Datos

El Backend API mantiene una dependencia directa con PostgreSQL/PostGIS para la persistencia transaccional y el aislamiento multi-tenant.

## 5.3. Backend y Motor de Ruteo

El Backend API se comunica con OSRM para obtener distancias y tiempos sobre la red vial, insumo clave para el motor de optimización VRP.

# 6. Contrato de una Operación

## 6.1. Operación

- **Método HTTP:** POST
- **Endpoint:** /api/routes/optimize
- **Descripción:** Ejecutar el cálculo de rutas óptimas para un conjunto de vehículos y entregas.

## 6.2. Cuerpo de Entrada

```
1 {  
2   "company_id": "uuid-empresa-123",  
3   "vehicle_ids": [  
4     "veh_01",  
5     "veh_02"  
6   ],  
7   "delivery_ids": [  
8     "del_101",  
9     "del_102",  
10    "del_103"  
11  ]  
12 }
```

## 6.3. Cuerpo de Salida

```
1 {  
2   "status": "success",  
3   "message": "Rutas calculadas correctamente",  
4   "routes": [  
5     {
```

```
6      "vehicle_id": "veh_01",
7      "sequence": [
8          "Bodega",
9          "del_101",
10         "del_103",
11         "Bodega"
12     ],
13     "total_distance_meters": 14200,
14     "total_duration_seconds": 3100
15 }
16 ]
17 }
```

## 7. ADR-001 – Selección de PostgreSQL con PostGIS

Se decide utilizar **PostgreSQL** con la extensión **PostGIS** como base de datos principal para centralizar la persistencia transaccional y el manejo de información geoespacial de manera integrada y eficiente bajo un esquema multi-tenant.

## 8. Walking Skeleton

El *Walking Skeleton* valida la arquitectura de extremo a extremo implementando el siguiente flujo vertical mínimo:

### 8.1. Paso 1 – Autenticación y configuración inicial

Un administrador inicia sesión en el sistema y registra una bodega central junto con un vehículo activo con capacidad máxima de **500 kg**, validando la conectividad entre el frontend, el backend y la base de datos PostgreSQL.

### 8.2. Paso 2 – Carga de entregas

El despachador accede a la vista de Administración y Despacho para cargar un archivo CSV con tres entregas de prueba. El backend valida la estructura y asocia los registros al inquilino (*tenant*) correspondiente.

### 8.3. Paso 3 – Geocodificación y ejecución del contrato de optimización

El sistema geocodifica los puntos de entrega. Al hacer clic en “**Calcular Rutas**”, el frontend invoca explícitamente el contrato HTTP POST en el endpoint `/api/routes/optimize`. El Backend procesa las coordenadas, consulta a OSRM, genera la matriz de distancias, ejecuta el motor VRP y persiste la ruta calculada.

### 8.4. Paso 4 – Visualización y retroalimentación en tiempo real por WebSockets

El conductor accede a la **Vista de Conductor** en su dispositivo móvil para visualizar la ruta asignada, el orden de paradas y un botón de navegación. Al actualizar el estado de una entrega (por ejemplo, marcada como “Entregada”), la aplicación emite un evento bidireccional mediante **WebSockets**, permitiendo que el panel del despachador refleje el cambio instantáneamente sin necesidad de recargar la página.

### 8.5. Flujo completo del Walking Skeleton

De esta manera, el Walking Skeleton permite comprobar la integración de los principales componentes arquitectónicos:

**Frontend Responsive → Backend API → PostgreSQL/PostGIS → OSRM → Motor VRP → Frontend Responsive**

Dentro del Frontend Responsive se utilizan las diferentes vistas según el rol del usuario:

**Administrador / Despachador → Vista de Administración y Despacho**

**Conductor → Vista de Conductor**

## 9. Resumen Arquitectónico

La arquitectura de RouteFlow separa limpiamente las responsabilidades mediante una aplicación web responsive, un Backend API modular basado en NestJS, persistencia transaccional y espacial en PostgreSQL/PostGIS, cálculo de ruteo con OSRM/OR-Tools y sincronización en tiempo real vía WebSockets.

## **Redefinición de la propuesta de valor**

**¿Para quien es el producto?:** Pequeñas y medianas empresas ya sea de distribución, e-commerce locales o tiendas que requieran hacer domicilios.

**¿Que necesitan estas empresas?:** Reducir el tiempo que toma calcular las rutas de forma manual o introducirlas una a una en google maps.

**¿Que hace nuestro saas?:** Sistema de gestión y optimización de rutas.

**¿En que nos diferenciamos?:** NO tienes que poner uno a uno el punto A al punto B en google maps o usar software a nivel empresarial de cuesta mas de 1000 USD/mes. Optimizamos la asignación de múltiples entregas para múltiples vehiculos en cuestión de minutos con un panel de control real.

## **Flujo critico**

**[1. Configuración de la flota] -> [2. Carga de Datos] -> [3. Optimización] -> [4. Ejecución] -> [5. Monitoreo]**

### **Paso a paso:**

**1- Configuración de la flota:** El administrador o quien despacha los paquetes registra los vehículos activos con su capacidad máxima de carga y el punto de origen (Desde donde salen).

**2- Carga de datos (Paquetes):** El administrador (Quien despacha) carga la lista de entregas del día (En formato csv o mediante un formulario simple pero estructurado) con: Dirección, peso del paquete y horario cuando salga.

**3- Optimización:** Se implementa el algoritmo de optimización y se usa cuando el usuario presiona: "Calcular ruta", ejecutando así en el backend la heurística de optimización, lo que genera algo así como Vehículo 1: Ruta A (Puntos ABC), Vehículo 2: Ruta B (Puntos DEF).

**4- Ejecución o Notificación:** La rutas ya optimizada le llega al conductor de forma organizada indicándole que camino tiene que seguir (Dirección A, Dirección B, Dirección C).

**5- Monitoreo:** El panel del administrador ve en tiempo real donde está el conductor y cuando marca las entregas como "Completadas" o "Fallidas".



## SI o SI

**1- Autenticación Multi-tenant básica:** Separar los datos de la Empresa A de la Empresa B.

**2- Integración con Motor Geográfico:** Convertir direcciones a coordenadas (Geocoding) y obtener matriz de tiempos/distancias.

**3- Algoritmo de Agrupamiento y Secuenciación:** La lógica matemática que decide el orden de las direcciones.

## Historias de usuario aprox

### ÉPICA 1: Configuración Inicial, Arquitectura e Infraestructura:

- HU-01: Inicialización del Proyecto y Entornos de Desarrollo:**

**Prioridad: Must Have**

**Como:** Desarrollador del equipo.

**Quiero:** Configurar el repositorio central (GitHub/GitLab) con la estructura base del proyecto (Monorepo o Frontend/Backend separados), entorno de Docker y linters/formateadores.

**Para:** Garantizar un flujo de trabajo ordenado, estandarizado y sin conflictos entre los 4 integrantes.

**Criterios de Aceptación:**

Repositorio creado con ramas principales (main, develop).

Archivo docker-compose.yml que levante el entorno local (Base de datos + Backend + Frontend).

Documentación inicial en README.md.

- **HU-02: Diseño y Creación del Esquema de Base de Datos Multi-tenant:**

**Prioridad:** Must Have

**Como:** Arquitecto de Base de Datos / Backend.

**Quiero:** Diseñar e implementar el modelo relacional de la base de datos (PostgreSQL / PostGIS).

**Para:** Soportar la arquitectura multi-tenant (aislamiento de datos por empresa) y permitir consultas espaciales/geográficas.

**Criterios de Aceptación:**

**Tablas creadas:** Empresas (Tenants), Usuarios, Vehiculos, Bodegas/Depósitos, Entregas/Paquetes, Rutas.

Claves foráneas e índices geográficos (PostGIS) correctamente definidos.

Ejecución exitosa de migraciones iniciales.

- **HU-03: Sistema de Autenticación y Autorización (JWT)**

**Prioridad:** Must Have

**Como:** Administrador o Usuario del sistema.

**Quiero:** Registrarme, iniciar sesión y recibir un token seguro (JWT).

**Para:** Acceder a la plataforma según mi rol (Administrador, Despachador, Conductor) y proteger los datos de mi empresa.

**Criterios de Aceptación:**

Endpoints /api/auth/register y /api/auth/login.

Contraseñas encriptadas con bcrypt.

Middleware que valide roles y restrinja accesos no autorizados.

## **ÉPICA 2: Gestión de Flota y Configuración de Bodega**

- **HU-04: Registro y Configuración de Bodega Principal (Punto de Origen)**

**Prioridad:** Must Have

**Como:** Administrador de Flota.

**Quiero:** Definir la dirección y coordenadas de mi bodega/depósito central.

**Para:** Que el motor de optimización sepa de dónde salen y a dónde regresan los vehículos.

**Criterios de Aceptación:**

**Formulario para ingresar dirección o seleccionar un punto en un mapa.**

**Geocodificación automática (conversión de dirección texto a latitud/longitud).**

**Persistencia del punto de origen vinculado al ID de la empresa.**

- **HU-05: Gestión de Vehículos de la Flota (CRUD de Vehículos)**

**Prioridad:** Must Have

**Como:** Administrador de Flota.

**Quiero:** Crear, listar, editar y desactivar vehículos de mi flota especificando su capacidad máxima (peso en kg y/).

**Para:** Que el sistema no asigne más carga a un vehículo de la que físicamente puede llevar.

**Criterios de Aceptación:**

**Formulario con campos:** Placa, Nombre/Modelo, Capacidad Máxima (Kg), Estado (Activo/Inactivo). Vista en tabla con filtrado de vehículos activos.

### **ÉPICA 3: Carga y Procesamiento de Entregas (Ingreso de Datos)**

- **HU-06: Carga Masiva de Entregas mediante Archivo CSV**

**Prioridad:** Must Have

**Como:** Despachador de Logística.

**Quiero:** Subir un archivo CSV/Excel con las entregas del día (cliente, dirección, peso del paquete, ventana horaria).

**Para:** Evitar digitar manualmente decenas o cientos de puntos de entrega.

**Criterios de Aceptación:**

Botón para importar CSV con validación de formato/columnas requeridas.

Notificación de errores en filas con datos inválidos o incompletos.

Inserción masiva de los registros válidos en la base de datos.

- **HU-07: Geocodificación y Validación de Direcciones en Mapa**

**Prioridad: Must Have**

**Como:** Despachador de Logística.

**Quiero:** Visualizar los puntos importados sobre un mapa interactivo para confirmar sus ubicaciones.

**Para:** Corregir manualmente coordenadas o direcciones mal formadas antes de calcular las rutas.

**Criterios de Aceptación:**

Integración con librería de mapas (Leaflet / Mapbox / Google Maps API).

Marcadores en el mapa para cada entrega.

Opción de arrastrar un marcador para ajustar la coordenada exacta si la dirección falló al geocodificarse.

#### **ÉPICA 4: Motor de Algoritmos y Optimización de Rutas (El Corazón del SaaS)**

Resuelve el problema matemático combinatorio de distribución de carga y secuencia de paradas.

- **HU-08: Integración con API de Matriz de Distancias y Tiempos**

**Prioridad: Must Have**

**Como:** Motor Backend de Optimización.

**Quiero:** Consumir una API de mapas (OSRM / Distance Matrix API) para obtener la matriz de tiempos y distancias reales entre todos los puntos.

**Para:** Alimentar el algoritmo con costos de traslado reales basados en la red vial.

**Criterios de Aceptación:**

Módulo que envíe la lista de coordenadas y retorne una matriz  $N \times N$  de distancias (metros) y tiempos (segundos).

Manejo de caché o almacenamiento temporal para no sobrecargar de peticiones la API externa.

- **HU-09: Algoritmo de Asignación y Secuenciación de Rutas (VRP Heurística)**

**Prioridad: Must Have**

**Como:** Despachador de Logística.

**Quiero:** Ejecutar el botón "Calcular Rutas Óptimas" para que el algoritmo procese los vehículos disponibles y las entregas pendientes.

**Para:** Obtener la distribución óptima que minimice la distancia total recorrida respetando los límites de carga por vehículo.

**Criterios de Aceptación:**

Implementación del algoritmo (Heurística de Clarke-Wright, Algoritmo Genético o biblioteca de optimización como OR-Tools).

Retorno de rutas agrupadas por vehículo, cada una con su secuencia ordenada de paradas (1ª parada, 2ª parada, etc.).

Tiempo de respuesta del algoritmo menor a 10 segundos para conjuntos de hasta 100 entregas.

## **ÉPICA 5: Despacho y Vista del Conductor**

- **HU-10: Vista Móvil / Lista de Hoja de Ruta para el Conductor**

**Prioridad:** Should Have (Si no hay app nativa, es una interfaz web responsiva)

**Como:** Conductor de la flota.

**Quiero:** Consultar en mi teléfono la lista ordenada de entregas asignadas a mi vehículo para el día.

**Para:** Saber exactamente a dónde ir, qué paquete entregar y en qué orden.

**Criterios de Aceptación:**

Vista responsiva (adaptada a pantallas móviles).

Botón directo en cada parada que abra la ubicación en Waze / Google Maps para la navegación paso a paso.

- **HU-11: Actualización de Estado de la Entrega (Completada / Fallida)**

**Prioridad:** Should Have

**Como:** Conductor de la flota.

**Quiero:** Marcar cada parada como "Entregada" o "Fallida" (con motivo de rechazo).

**Para:** Actualizar el estado del despacho en tiempo real hacia el centro de control.

**Criterios de Aceptación:**

Botones de acción rápida por cada entrega.

Registro de la hora exacta (timestamp) en que se cambió el estado.

## **ÉPICA 6: Monitoreo en Tiempo Real y Métricas**

- **HU-12: Dashboard de Seguimiento de Flota (WebSockets / Polling)**

**Prioridad:** Should Have

**Como:** Despachador / Administrador.

**Quiero:** Un panel de control central con un mapa que muestre el avance de las rutas y el estado de cada entrega en vivo.

**Para:** Detectar retrasos y supervisar la operación del día sin llamar por teléfono a los conductores.

**Criterios de Aceptación:**

Actualización en tiempo real mediante WebSockets o polling.

Cambios de color en los marcadores del mapa según el estado (Azul = Pendiente, Verde = Completado, Rojo = Fallido).

Indicador de porcentaje de progreso global del día (% de entregas completadas).